Computers & Graphics 141 (2026) 104811

Contents lists available at ScienceDirect

Computers & Graphics

journal homepage: www.elsevier.com/locate/cag

Special Section on CAD/Graphics 2026

From objects to agents: Object-level reinforcement learning for 3D indoor scene synthesis☆

Yunsik Cho [a], Jiwon Lee [a], Jinmo Kim [a,b,*]

[a] Department of Information and Computer Engineering, Hansung University, 116 Samseongyo-ro 16-gil, Seongbuk-gu, Seoul, 02876, Republic of Korea
[b] Division of Computer Engineering, Hansung University, 116 Samseongyo-ro 16-gil, Seongbuk-gu, Seoul, 02876, Republic of Korea

ARTICLE INFO

Keywords:
3D indoor scene synthesis
Object-level reinforcement learning
Shared policy
Semantic relationships
Scalability

ABSTRACT

Automated 3D indoor scene synthesis is the problem of arranging multiple furniture objects within a given space while simultaneously satisfying geometric constraints based on object–object and object–layout relations, as well as semantic interpretations of these relationships. In this work, we define scene synthesis as an object-level reinforcement learning problem, where the scene is reconstructed not as a global state interpretation, but as a collection of object-wise decision-making processes. Specifically, each furniture object is established as a one-to-one correspondence structure of an independent decision-making agent. The proposed method applies a shared policy to diverse room layouts and object combinations. We categorize scene objects according to their characteristics and learn placement policies through reward that simultaneously achieves physical collision avoidance and preservation of inter-object relationships. Based on this, we construct a flexible framework capable of generating complex and diverse scenes across various layouts, room types, and object configurations. Furthermore, we systematically validate the accuracy of scene generation in terms of geometric constraints and semantic consistency, as well as the scalability of the proposed method, through both quantitative and qualitative experiments. The results confirm that object-level policy learning forms satisfactory placement strategies under diverse conditions and demonstrates improved performance and scene generation quality.

1. Introduction

3D indoor scene synthesis is the problem of generating scenes in which multiple furniture objects coexist within a space while simultaneously satisfying geometric constraints, such as physical collision avoidance, and semantic constraints, such as relational placement between associated objects (e.g., desk-chair configurations). Such technology is widely used in various application domains, including virtual reality, augmented reality, architectural visualization, and digital content production, where indoor spaces must be efficiently organized and diverse scene scenarios automatically constructed. Recently, diverse studies have addressed indoor scene synthesis from different perspectives. Optimization-based approaches define cost functions that encode geometric and semantic relationships among objects and infer natural placements by minimizing these objective functions [1–3]. Data-driven generative models learn object placement distributions from large-scale scene datasets and generate visually plausible scenes accordingly [4–8]. Reinforcement learning-based approaches formulate scene layout as a sequential decision-making problem and progressively improve object arrangements toward desired objectives [9,10]. These research directions apply different design strategies to the scene synthesis problem and present diverse perspectives, including constraint satisfaction, distribution learning, and sequential optimization.

In this work, we propose a flexible framework that goes beyond limited scene and object combinations and enables scene synthesis under diverse conditions. The proposed method induces accurate results that stably satisfy geometric constraints and semantic interpretations regarding object–object and object–layout relations. Furthermore, we systematically address scalability so that the framework can be extended to industrial and academic applications.

To this end, we reinterpret 3D indoor scene synthesis from an object-level reinforcement learning perspective. All furniture objects composing a scene are defined as independent decision-making agents, which interact simultaneously and gradually form a scene layout through repeated actions. Each agent learns a placement policy based on its local observation and reward signals derived from its characteristics, aiming to satisfy geometric and semantic relationships simultaneously. Once trained, the learned policy can synthesize accurate and novel scenes from diverse layouts and object combinations.

☆ This article was recommended for publication by Junhui Hou.

* Corresponding author at: Division of Computer Engineering, Hansung University, 116 Samseongyo-ro 16-gil, Seongbuk-gu, Seoul, 02876, Republic of Korea.
E-mail addresses: yunsik.cho@hansung.ac.kr (Y. Cho), mpm11@hansung.ac.kr (J. Lee), jinmo.kim@hansung.ac.kr (J. Kim).

https://doi.org/10.1016/j.cag.2026.104811
Received 9 June 2026; Received in revised form 8 October 2026; Accepted 13 October 2026
Available online 21 October 2026
0097-8493/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Moreover, large numbers of scenes can be generated in parallel under a shared policy, demonstrating that the proposed framework can function not only as a single-scene synthesis method but also as a scalable scene generation tool. The main contributions are summarized as follows:

1. **Flexibility:** We propose a flexible scene synthesis framework that is not restricted to limited scene and furniture combinations, and enables a shared policy to be applied across diverse layouts, room types, and object configurations.
2. **Accuracy:** Based on placement policies that simultaneously achieve geometric collision avoidance and semantic relationship preservation, the proposed method produces accurate and high-quality scene synthesis results.
3. **Scalability:** Through parallel scene inference, multiple independent scenes can be generated efficiently. This provides a new direction that alleviates the limitations of previous approaches that are heavily dependent on fixed datasets.

In this study, we conduct systematic quantitative and qualitative experiments to validate these contributions. In addition to qualitative visualizations of various generated scene, we evaluate scene quality and diversity using the Fréchet Inception Distance (FID), verify geometric correctness through oriented bounding box (OBB)-based scene accuracy, and perform comparative performance analysis. Through these experiments, we aim to demonstrate the novelty of the proposed framework in terms of flexibility, accuracy, and scalability.

2. Related work

Research on 3D indoor scene synthesis can be broadly categorized into three directions: optimization-based approaches, data-driven deep generative models, and reinforcement learning-based methods. Each approach interprets the scene synthesis problem from a different perspective.

2.1. Scene optimization

Early studies in automated scene synthesis formulated furniture arrangement as a constrained optimization problem defined by cost functions. Geometric constraints and semantic interpretations — such as accessibility, inter-object distances, and visibility — were encoded into cost functions, and layouts were explored by searching for solutions that minimize these objectives. Yu et al. [1] optimized furniture placement using simulated annealing and Metropolis–Hastings-based stochastic search. Subsequent research sought to improve computational efficiency, including genetic algorithm-based multi-candidate optimization [2] and greedy cost minimization techniques [11]. Weiss et al. [3] proposed a continuous optimization framework that iteratively satisfies constraints using a physical spring model and position-based dynamics. While these methods excel at inducing stable results through explicit rule representation, they generally treat the entire scene as a single global cost function. The layout is computed by searching the relations among all furniture objects in the space at every step. Consequently, the layout cost must be evaluated at each step of the furniture placement process.

2.2. Deep learning-based generative models

With the emergence of large-scale 3D scene datasets, deep learning-based generative models — which directly learn object placement distributions from data — have become a dominant research trend. Yang et al. [4] employed a VAE-based architecture to jointly predict absolute and relative object attributes. Paschalidou et al. [5] introduced an autoregressive transformer structure that models scenes as unordered object sets, while Tang et al. [6] adopted a diffusion model to concurrently refine object attributes. Earlier deep generative pipelines, including the convolutional top-down framework of Ritchie et al. [12], used learned image-based priors to insert objects iteratively. Building on diffusion-based synthesis, Yang et al. [13] further integrated physics- and interactivity-based guidance to enforce collision avoidance and reachable layouts for embodied AI scenarios. More recently, approaches leveraging large language models (LLMs) to generate spatial relationships or construct placement constraints have been proposed [7,8], with subsequent work extending this direction to vision-language models for differentiable layout optimization [14] and to graph-augmented diffusion that accepts free-form text and image inputs [15]. Although these generative models successfully produce scenes with high visual quality, they are fundamentally constrained by the distributions present in the training datasets. Furthermore, they do not inherently guarantee the avoidance of geometric collisions or the fulfillment of functional placement conditions, often necessitating post-processing steps to align the generated results with specific constraints. These limitations have led to an alternative perspective: iteratively improving scene synthesis based on target conditions.

2.3. Reinforcement learning

Recently, several studies have formulated scene synthesis as a sequential decision-making problem to be solved via reinforcement learning (RL). In the related task of scene rearrangement [16–18], Wang et al. [16] learn move-planning policies between given initial and target layouts via deep RL combined with Monte Carlo tree search. Ostonov et al. [9] generate scenes through a hierarchical RL structure that adds objects one by one, while Sun et al. [10] optimize pre-arranged scenes by selecting and relocating furniture based on human-centered objectives using an RL-based approach. The advantage of these methods lies in their ability to progressively refine scenes by incorporating geometric constraints and human-centric placement conditions into reward functions. However, existing research considers only sequential placement based on a single global policy, and does not consider the process in which multiple objects are placed simultaneously. In this study, the furniture objects composing a scene are defined as agents, and each is designed as an independent agent capable of making its own decisions, so that the scene is synthesized by performing the furniture placement actions simultaneously.

Multi-agent reinforcement learning of the proximal policy optimization (PPO) family includes methods in which each agent learns independently while treating the other agents as part of the environment [19], methods in which learning uses global information while observation and action are executed with respect to the agent itself [20], and fully centralized methods in which the actions of all agents are directed from a single center [21]. Building on prior work demonstrating that independent learning alone can achieve improved performance, this study proposes object-level reinforcement learning for indoor scene synthesis. We proceeded on the judgment that each furniture agent, which regards the room structure and the other furniture agents being placed as part of its own placement conditions, can achieve effective performance in scene synthesis even when it learns independently.

Table 1 analyzes the characteristics and limitations of prior work on scene synthesis and, on that basis, derives the research directions and hypotheses for applying reinforcement learning.

3. Overview

In this work, we interpret the 3D indoor scene synthesis problem from an object-level reinforcement learning perspective. Specifically, all furniture objects composing a scene are mapped to individual agents, establishing a one-to-one structure between objects and agents. Each agent learns an action policy consisting of translation and rotation, aiming to maximize rewards defined based on its local observations. As a result, all furniture objects iteratively adjust their positions and orientations according to their surrounding environment.

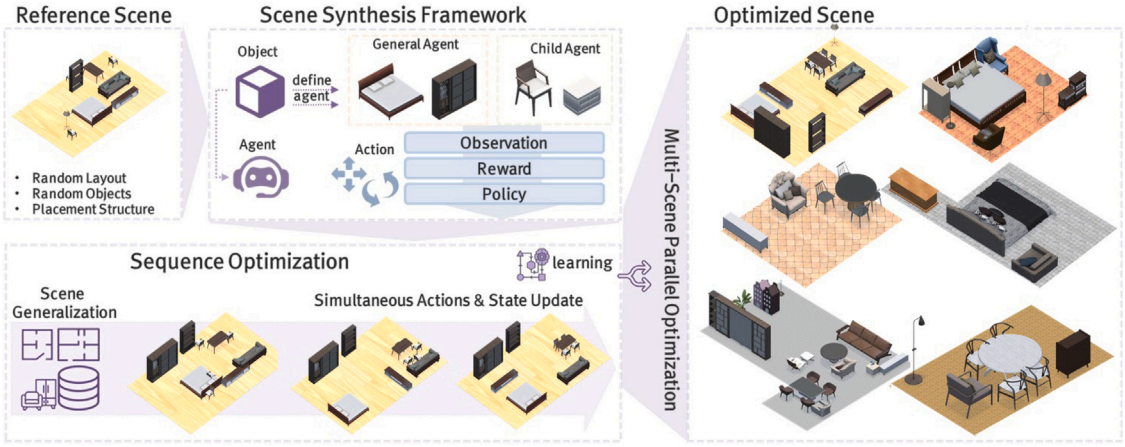


Fig. 1. Overview of the proposed object-level reinforcement learning-based scene synthesis framework.

Table 1
Characteristics and limitations of prior work on scene synthesis, and the research directions and implications of applying reinforcement learning.

	Optimization-based	Generative model-based
Characteristics	• Explicit cost function proposed for object placement	• Fast placement inference through a learned model
Limitations	• Convergence to local minima • Repeated search computation	• Results limited to the dataset • Post-processing required for correction
Research directions and implications of applying reinforcement learning	• Minimizing the repeated computation process through observation and reward function design • A learning and inference structure that can rapidly generate a large number of scenes	

Through repeated interactions, the agents collectively infer a scene layout that simultaneously satisfies geometric and semantic relationships. Fig. 1 illustrates the overall architecture of the proposed object-level reinforcement learning-based scene synthesis framework.

4. Object-level reinforcement learning

4.1. Formulation

A scene consists of N furniture objects, each mapped to an individual agent (i). At time step t, the scene state is denoted as S_t, which includes the positions and spatial relationships of all objects. At the same time step t, each agent i observes its local observation o_t^i, computed from its own state and surrounding environment (room layout and neighboring agents), and selects an action a_t^i according to its policy π_θ. Once the actions of all agents are applied to the scene state, the environment transitions to the next state S_{t+1} by enforcing layout boundary constraints and inter-object collision constraints. This interaction continues for at most T steps, and a new episode begins when a termination condition is satisfied. Through this process, the accumulation of local actions from individual agents results in the formation of a global scene layout. The objective of training is to maximize the expected cumulative reward with respect to the policy π, as defined in Eq. (1).

$$\mathbb{E}_{\pi_\theta}\left[\sum_{t=1}^{T}\gamma^t\sum_{i=1}^{N}R(S_t^i,a_t^i)\right] \tag{1}$$

where $\mathbb{E}$ denotes the expectation over the trajectory distribution induced by the policy π_θ and environment transitions, $\gamma \in [0,1]$ is the discount factor that determines the relative importance of future rewards, and $R(S_t^i,a_t^i)$ represents the reward computed from action a_t^i of agent i at time step t given local state S_t^i.

4.2. Agent design

In this work, all furniture objects composing a scene are mapped to individual agents. However, not all objects share the same placement objectives. For example, general objects such as desks or cabinets primarily aim to maintain appropriate distance and orientation alignment with respect to walls, while avoiding collisions. In contrast, within hierarchical relationships such as a desk-chair pair, a child object (e.g., a chair) is influenced more strongly by its relative placement with respect to its parent object (e.g., a desk) than by its direct relation to the room layout. That is, this is expressed as a parent–child relationship. In this case, the child object (e.g., a chair) is placed by following only the objective with respect to its parent object (e.g., a desk). Since the required learning objectives differ according to object characteristics, we classify agents accordingly. Each agent belongs either to the set $\mathcal{P}$ (which may include parent objects) or to the set C of child objects. For each child object, a corresponding parent object is predefined, and the learning objective is to maintain relative distance and orientation with respect to its parent. In contrast, a general agent aims to align with the room layout in terms of distance and orientation while avoiding collisions with other agents. Accordingly, each agent is designed to have a different internal state representation s_i, which is defined as s_i^P for general agents and s_i^C for child agents. Fig. 2(a) illustrates the conceptual structure of the proposed object-to-agent mapping and agent design.

4.3. Observation representation

The observation of agent i, denoted as o_i, is composed of the role-specific internal state s_i and the ray-based environmental observation r_i, and is defined as:

$$o_i = [s_i; r_i] \tag{2}$$

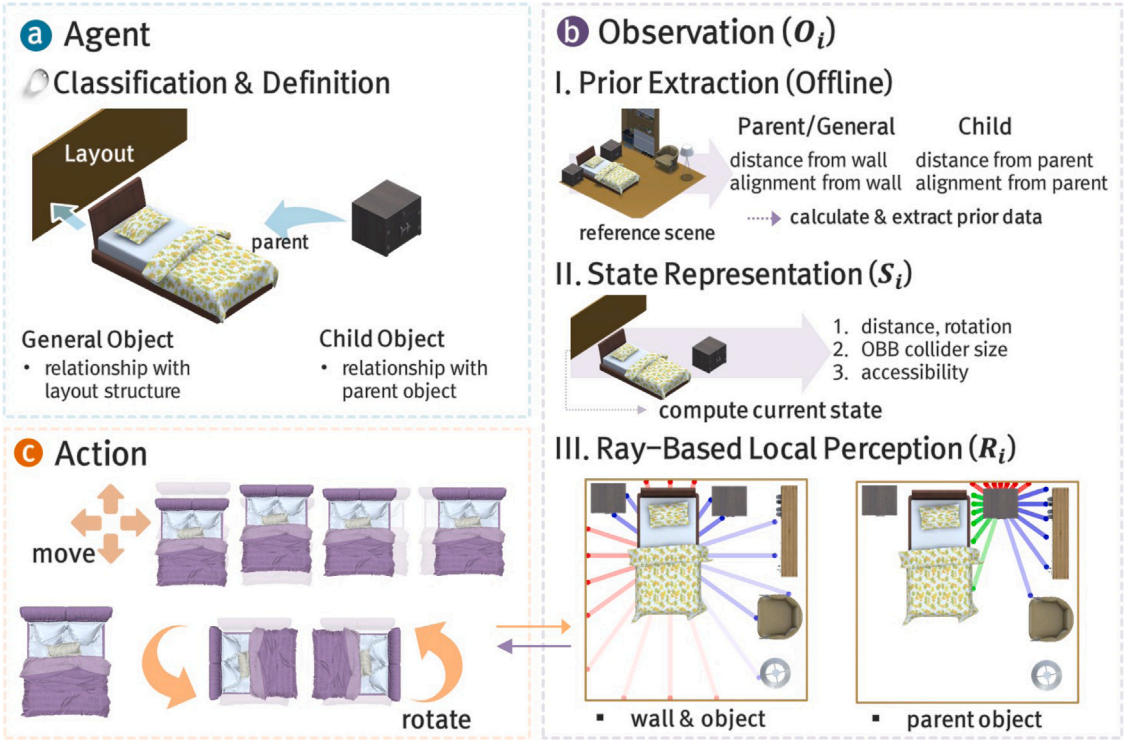


Fig. 2. The proposed agent design. (a) general and child objects have different state representations. (b) the observation is composed of the object-specific internal state s_i and ray-based environmental observation r_i. (c) at each decision step t, the agent selects either a translation or rotation action.

Here, s_i includes both predefined target reference values and currently observed relational values. As described in Section 4.2, for a general agent, s_i contains the target distance to the wall, the current distance to the wall, orientation alignment with respect to the layout, and collision status. For a child agent, s_i includes the target distance to the corresponding parent agent, relative orientation alignment, and relative position information defined in the parent's coordinate frame. In addition, all agents commonly include object size information representing their physical extent. This is essential for learning geometric constraints such as collision avoidance and space reservation. While s_i summarizes the target relations and current relational states, r_i provides environmental observation information for perceiving the surrounding spatial structure that dynamically changes during placement. Fig. 2(b) illustrates the ray-based environmental observation mechanism. The ray-based observation r_i is defined in the local coordinate frame of the agent. From its current position, each agent uniformly emits K rays over 360° on the X-Z plane. Each ray explores the surrounding environment up to a maximum detection distance L. Through this process, the agent obtains normalized distance information to walls and neighboring furniture objects. For child agents, the corresponding parent object is additionally detectable within the ray observations. In our implementation, we set $K = 20$ and $L = 5$.

4.4. Agent action

In this work, all agents are assumed to be placed on a single-layer flat floor. Agent translation is restricted to the X-Z plane, and rotation is defined around the vertical axis of the world coordinate system (yaw). The rotation is discretized into 90° increments rather than continuous values. This design reflects the common characteristic of indoor furniture placement, where objects are typically aligned orthogonally with walls; from an analysis of the bedroom, dining room, and living room scenes of 3D-FRONT [22], we confirmed that approximately 95.4% of the object orientations lie within 1° of a 90° increment. Accordingly, at each decision step, agent i selects one action from the set $\mathcal{A}_i$ of seven actions, which consists of keeping the current position and orientation, translating by δ in either direction along its own forward axis $\mathbf{f}_i$ or right axis $\mathbf{r}_i$, and rotating by -90° or +90°.

$$\mathcal{A}_i = \{ \varnothing,\ +\delta\,\mathbf{f}_i,\ -\delta\,\mathbf{f}_i,\ +\delta\,\mathbf{r}_i,\ -\delta\,\mathbf{r}_i,\ -90^\circ,\ +90^\circ \} \tag{3}$$

Defining the actions in this simple form reduces the branching factor and improves learning stability. At each decision step t, all agents simultaneously select their actions based on their local observations. After actions are applied, the environment computes the state transition by enforcing layout boundary constraints and inter-object collision constraints. Invalid movements (e.g., actions that move an object outside the valid floor region) are corrected according to predefined constraint rules. Based on the updated state, new observations and rewards are generated. Fig. 2(c) illustrates this action execution and state transition process.

4.5. Reward

The reward serves as a learning signal defined based on the outcome of each agent's action, guiding the agent to progressively satisfy its corresponding objectives. At time step t, the reward received by a general agent P_i and that received by a child agent C_i are defined as follows:

$$r_P^{i,t} = r_{\text{acc}}^{i,t} + r_{\text{prior}}^{i,t} \tag{4}$$

$$r_C^{i,t} = r_{\text{acc}}^{i,t} + r_{\text{pair}}^{i,t} \tag{5}$$

The relative importance among the accessibility, prior, and pairwise rewards is controlled by reward-component weights w inside each reward. For notational simplicity, we assume that all reward terms are computed at the same time step for object i, and we omit the time index t and agent index i when no ambiguity arises.

The reference values of the prior and pairwise terms in Eqs. (4) and (5) are computed from a reference scene, and the reference relations are set separately. One reference scene is determined for each scene

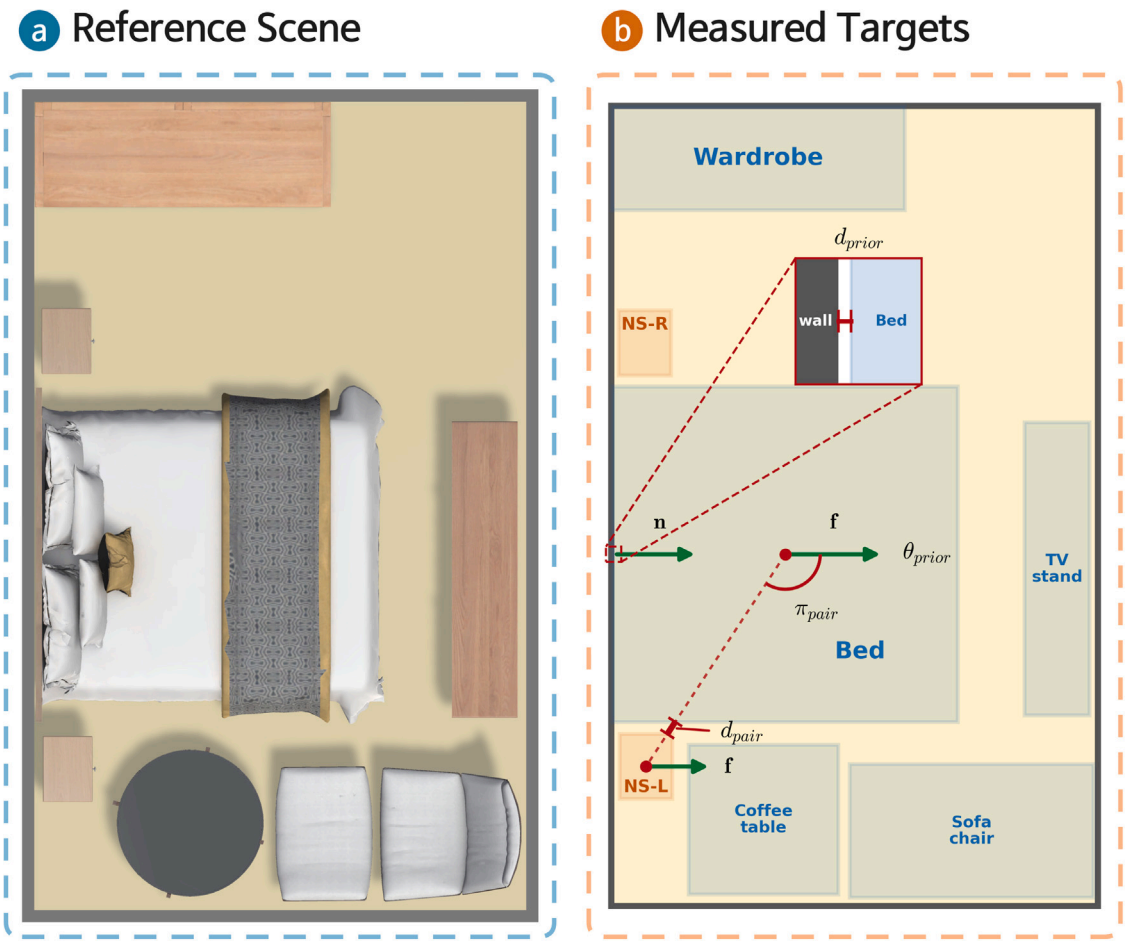


Fig. 3. The bedroom reference scene and the reference values computed from it. (a) Orthographic top view. (b) OBB representation of the same room: blue denotes general agents, orange denotes child agents, and green denotes the forward direction of an object or the normal vector of a wall pointing into the room. For the bed, d_{prior} denotes the distance between the object surface and the nearest wall surface, and θ_{prior} denotes the inner product between the forward direction vector **f** of the object and the vector **n** of the wall pointing into the room. d_{pair} denotes the shortest surface distance along the line from the center of the parent object to the center of the child object in a parent–child relation, and π_{pair} denotes the angle between that line and the forward direction vector **f** of the parent object. The measured values are summarized in Table 2.

category (bedroom, living room, and dining room). From the 3D-FRONT dataset [22] we manually select a room that contains all of the categories used for furniture placement and whose objects are aligned with the walls in 90° increments, that is, the inner product between the forward direction vector **f** of an object and the inward normal vector **n** of the wall is −1, 0, or 1. Fig. 3 shows an example: for the bedroom reference, we select a scene that contains all furniture categories used for placement, including a bed, wardrobe, TV stand, and nightstand, and whose object orientations satisfy the condition above. However, since the dataset has no parent–child relation information, we distinguish general objects from child objects and assign a parent to each child object, thereby defining the reference relations. In the dining room and the living room, chairs were assigned to the dining table and armchairs to the sofa, and in the bedroom, nightstands were assigned to the bed. The placement targets are not averaged over several rooms or drawn from a distribution; they are the clearance distance, angle, and orientation values measured from the positions and orientations of the objects placed in this single room. Because these values are independent of object size, they remain valid even when the furniture model changes from episode to episode or the room size differs, and accordingly a single reference scene per scene category is sufficient for all episodes. Table 2 shows the reference values and the reference relations derived from the reference scene.

4.5.1. Accessibility

The reward r_{acc}, which is commonly applied to all agents, encourages collision avoidance among objects. If an agent's translation or rotation results in a collision with another object, a penalty is assigned. Conversely, when a collision state is resolved, a positive reward is provided. To determine collision status, we adopt a discrete collision detection scheme based on the intersection of OBBs on the X-Z plane, considering implementation simplicity and learning stability. The box is the tightest rectangular volume that contains the object mesh, but it is still wider than the actual shape. Since an object with an irregular shape is also treated as occupying the entire extent of the box, a collision can be detected even when the actual meshes do not overlap. The accessibility reward is defined as follows:

$$r_{\text{acc}} = -w_{\text{acc}}\, C_i \tag{6}$$

Here, C_i is 1 if agent i overlaps an object that is neither its parent nor its child, and 0 otherwise. Since an overlapping placement can itself be the target for a parent–child pair (Section 4.5.3), the overlap between such a parent–child pair is excluded from the penalty. All other pairs, that is, between general objects, between child objects, and between a child and a general object that is not its parent, are penalized as before.

4.5.2. Prior (General agent)

A general agent learns a structurally aligned placement with respect to the wall. The reward r_{prior} is defined based on the target distance

Table 2
Example of the reference values computed from the bedroom reference scene and the assigned reference relations.

Object	Agent	Parent	d_{prior} (m)	θ_{prior}	d_{pair} (m)	π_{pair} (°)
Bed	general	–	0.010	1.000	–	–
Cabinet (wardrobe)	general	–	0.003	1.000	–	–
Cabinet (TV stand)	general	–	0.052	1.000	–	–
Table (coffee table)	general	–	0.072	1.000	–	–
Sofa (sofa chair)	general	–	0.050	0.000	–	–
Nightstand (NS-L)	child	Bed	–	–	0.089	+123
Nightstand (NS-R)	child	Bed	–	–	0.083	−124

to the wall and orientation alignment, and is designed to converge toward reference values pre-extracted from a reference scene. During training, the agent observes its current distance to the wall and the degree of orientation alignment. A higher reward is assigned as these values approach the predefined reference values. The prior reward is defined as follows:

$$r_{\text{prior}} = -w^{d}_{\text{prior}} \left| d - d_{\text{prior}} \right| - w^{\theta}_{\text{prior}} \left| \theta - \theta_{\text{prior}} \right| \tag{7}$$

Here, d and θ denote the distance to the wall and the rotation value observed by the object in the current scene, respectively. d_{prior} and θ_{prior} represent the ideal distance and orientation with respect to the wall, extracted from the reference scene. d_{prior} is the clearance distance between the OBB of the object and the nearest wall along the object's own axis. θ_{prior} is the cosine similarity between the forward axis $\mathbf{f}$ of the object and the inward normal $\mathbf{n}$ of that wall.

4.5.3. Pairwise relationship (Child agent)

A child agent learns primarily based on its relative placement relationship with the corresponding parent object. The reward r_{pair} is defined based on the distance to the parent, orientation alignment, and relative positional relationship defined in the parent's coordinate frame. All reference values are computed on the basis of the parent–child relations set in the same reference scene described in Section 4.5. The child agent learns a policy that progressively converges toward these predefined relational targets. The pairwise relationship reward is defined as follows:

$$\begin{aligned} r_{\text{pair}} = & -w^{d}_{\text{pair}} \left| d - d_{\text{pair}} \right| \\ & - w^{\theta}_{\text{pair}} \left| \theta - \theta_{\text{pair}} \right| \\ & - w^{\pi}_{\text{pair}} \left\| \pi - \pi_{\text{pair}} \right\| \end{aligned} \tag{8}$$

Here, d, θ, and π denote the current distance, orientation alignment, and relative position between the child agent and its corresponding parent agent, respectively. d_{pair}, θ_{pair}, and π_{pair} represent the reference distance, orientation alignment, and relative position between the child and parent extracted from the reference scene. The orientation term θ is computed as the cosine similarity between the unit forward direction vector of the child agent and the unit direction vector $u = \frac{\text{pos}(\text{p}(i)) - \text{pos}(i)}{\|\text{pos}(\text{p}(i)) - \text{pos}(i)\|}$, where $\text{pos}(i)$ denotes the 3D position of agent i, and $\text{p}(i)$ indicates the parent object of agent i. Thus, u points from the child agent to its parent. Here, $\text{pos}(i)$ is the OBB center of object i, and $\text{p}(i)$ is the parent object of object i. Only the X and Z coordinates of $\text{pos}(i)$ are used in the reward computation; that is, the height is not considered and $\text{pos}(i)$ is treated as a 2D position on the floor. π is the unit direction from $\text{pos}(\text{p}(i))$ to $\text{pos}(i)$, expressed in the coordinate frame of the parent and restricted to the X–Z plane. Since π carries no distance information, the three terms of Eq. (8) are non-redundant and each is responsible for one quantity: d_{pair} constrains how far the child is from the parent, π_{pair} constrains in which direction of the parent it lies, and θ_{pair} constrains which way the child faces. d_{pair} is the signed gap measured between the two box surfaces along the line connecting $\text{pos}(\text{p}(i))$ and $\text{pos}(i)$ (Fig. 3). A parent–child relation such as desk–chair involves adjacency or partial overlap, and in such an interpenetrating placement d_{pair} becomes negative.

5. Scene generalization

To prevent overfitting to limited furniture objects or fixed layouts and to learn placement policies that are applicable across diverse scene conditions (e.g., room types, object combinations, and layouts), we propose a generalization strategy applied during episode initialization.

1. We determine the scene category for which the placement is to be performed.
2. A scene layout randomly extracted from the 3D-FRONT dataset [22], together with the wall information surrounding it, is given as input.
3. From the 3D-FUTURE dataset [23], one model per category is selected to compose the list of furniture data to be placed.
4. The list of furniture data is placed at random positions within the layout boundary, with a random rotation value in 90° increments.

Starting from the initial scene configuration set in this way, the scene is completed by taking the reference values and the reference relations into account (see Fig. 4).

6. Training configuration

The proposed object-level reinforcement learning framework was implemented and trained in a Unity-based simulation environment [24]. Reinforcement learning was implemented using the Unity ML-Agents Toolkit [25], and PPO [26] was adopted as the policy learning algorithm. PPO is a standard policy gradient method that suppresses abrupt policy updates through a clipping mechanism and exhibits stable convergence properties under the defined discrete action space. Since general agents and child agents have different characteristics, separate policy networks were constructed for the two agents. While both policies share the same PPO hyperparameters, their observation vectors and reward structures are defined differently according to their respective characteristics. The main hyperparameters used for PPO-based policy learning, adopted from the default configuration provided by the ML-Agents Toolkit, are summarized in Table 3. The reward-component weights w in Eqs. (6)–(8) were tuned manually starting from a common initial value, with adjustments guided by ablation results; the final values are $w_{\text{acc}} = 0.02$, $w^{d}_{\text{prior}} = 0.03$, $w^{\theta}_{\text{prior}} = 0.01$, $w^{d}_{\text{pair}} = 0.03$, $w^{\theta}_{\text{pair}} = 0.01$, and $w^{\pi}_{\text{pair}} = 0.01$. The policy network consists of a three-layer multilayer perceptron (MLP) with 512 hidden units per layer. The discount factor was set to $\gamma = 0.99$, the learning rate to 3×10^{-4}, and the GAE parameter to $\lambda = 0.95$. The total training process was conducted for 1.5×10^{7} steps.

Furniture object models were obtained from the 3D-FUTURE dataset [23], and the room layouts sampled at episode initialization were obtained from the 3D-FRONT dataset [22]. The reference scene from which the placement targets are measured is an individual room of the same dataset, selected one per scene category as described in Section 4.5. Fig. 5 illustrates the progression of a single episode. At each decision step, all agents simultaneously select their actions. The environment then updates the scene state by enforcing layout boundary constraints and inter-object collision constraints. New observations and rewards are subsequently computed. Through repeated interactions, the

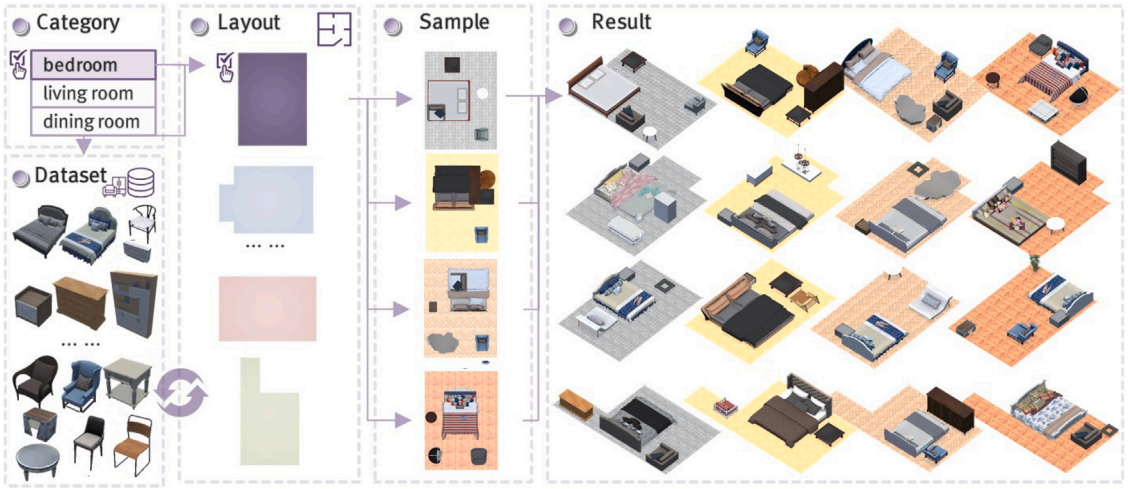


Fig. 4. Scene generalization through random object combinations based on scene categories and layout patterns.

Table 3
The proximal policy optimization (PPO) hyperparameters used for training.

Parameter	Value
Batch size	64
Buffer size	409,600
Learning rate	3×10^{-4}
Discount factor (γ)	0.99
GAE (λ)	0.95
Clip parameter	0.2
Entropy coefficient	5×10^{-3}
Num epoch	3
Time horizon	64
Max steps	1.5×10^{7}
Hidden units	512
Num layers	3

layout gradually stabilizes. An episode terminates when all agents satisfy their respective objectives within a predefined tolerance threshold or when the maximum number of steps is reached, resulting in the generation of a finalized scene.

7. Experiments and analysis

The proposed object-level scene synthesis framework is designed with three primary objectives: flexibility, accuracy, and scalability. To validate these core objectives, we conduct systematic quantitative and qualitative experiments. First, to verify flexibility, we analyze whether the learned policy can generate natural scenes composed of complex furniture combinations under diverse layout conditions. For this purpose, we perform distribution similarity analysis and qualitative scene visualization experiments to assess the degree of satisfaction in generated scene synthesis. Next, we conduct stepwise comparative experiments to systematically analyze accuracy based on geometric and semantic constraints. Finally, to explore a new direction regarding scalability, we construct a parallel inference environment composed of multiple independent scenes using the trained policy. We design experiments that evaluate throughput, including scene synthesis time.

7.1. Flexibility

To analyze how flexibly the proposed framework generalizes under diverse scene conditions, we conduct quantitative comparison experiments using the Fréchet Inception Distance (FID), which evaluates the quality and diversity of generated scene distributions. As comparison baselines, we select ATISS [5], a representative autoregressive generative model–based approach, and LayoutGPT [7], a recent method leveraging LLMs, and DiffuScene [6], a diffusion model–based approach. DiffuScene is evaluated in the scene rearrangement setting. Since the model generates the positions and orientations while the object set of each evaluation scene is preserved, it is under the same fixed object set condition as the proposed method. However, the released model trained for the rearrangement setting is not conditioned on the floor boundary. ATISS, LayoutGPT, and DiffuScene all report FID over rendered scenes in their own evaluations, and our FID measurement follows this practice. In addition, without directly learning the scene data distribution, the observation and reward design of reinforcement learning-based scene synthesis makes it possible to obtain diverse scene results that still follow the distribution of the dataset. Existing methods typically learn scene distributions separately for each scene category (e.g., bedroom, living room, dining room), or explicitly condition the generation process on scene type through input prompts. As a result, the training process or generation condition is clearly separated according to scene categories. In contrast, the proposed method does not rely on separate policy training or explicit category conditioning. Instead, a unified object-level policy is trained under a shared structure without distinguishing scene types. The experiments utilize three scene categories from the 3D-FRONT dataset [22]. Generated scenes are rendered as top-view images under identical camera and lighting conditions. FID is computed against a reference distribution formed from the 3D-FRONT rooms themselves. The reference rooms are rendered with the same renderer as the generated layouts, so that the comparison reflects differences in placement rather than differences in rendering. The per-category reference set is the entire 3D-FRONT collection of that room type (4041 bedrooms, 987 living rooms, and 900 dining rooms). Each method generates the number of images listed in Table 4. When the evaluation set of a method contains fewer rooms than this number, the same room is used several times to produce different samples. The public implementation of LayoutGPT that we evaluated provides prompts and data only for the bedroom and the living room, so we do not report dining room results. This is a matter of the released configuration, not a limitation of the method itself. For DiffuScene as well, the released model trained for the rearrangement setting covers only the bedroom and the living room, so the dining room entry is left empty for the same reason.

As shown in Table 4, The proposed method achieves FID scores that are comparable to or lower than those of existing approaches across all categories. In particular, it maintains stable performance in the living room category, where a relatively larger number of objects and more complex interactions must be considered. This result suggests that

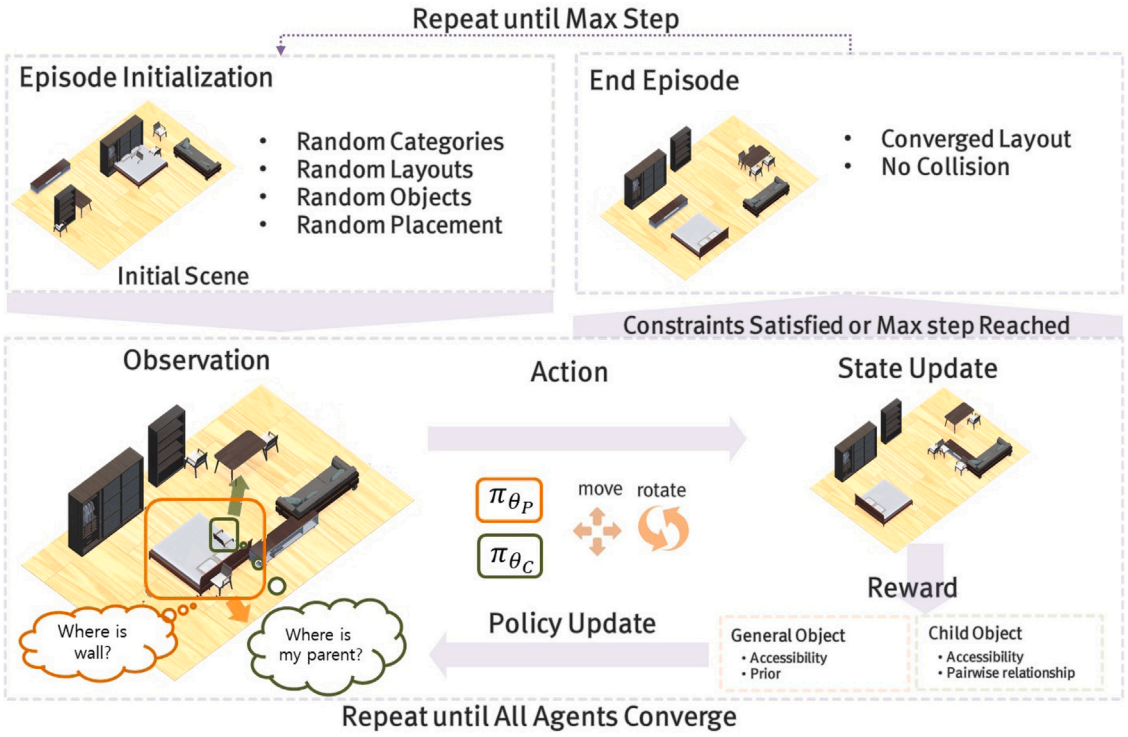


Fig. 5. Training episode progression of the object-level reinforcement learning-based scene synthesis framework. Through parallel action selection by agents and the application of environmental constraints, the scene layout is iteratively refined, and a final scene is generated upon episode termination.

Table 4
Quantitative comparison of FID scores for bedroom, living room, and dining room scenes from the 3D-FRONT dataset [22] among ATISS [5], LayoutGPT [7], DiffuScene [6], and the proposed method. Each method's generated images (count in parentheses) are compared against the entire 3D-FRONT set of the same room type — 4041 bedrooms, 987 living rooms, and 900 dining rooms.

Category (number of images)	ATISS [5]	LayoutGPT [7]	DiffuScene [6]	Ours
Bedroom (423)	67.254	69.717	72.830	**66.196**
Living Room (53)	142.304	169.110	157.481	**125.929**
Dining Room (177)	102.768	–	–	**98.571**

even without category-specific distribution design, the accumulation of object-level decision-making processes can produce layouts whose image-level distribution, as measured by FID, is close to that of the reference scenes. These findings suggest that the proposed framework can adapt to diverse scene conditions using a unified policy structure without relying on category-wise distribution learning, thereby supporting the structural flexibility of the object-level reinforcement learning-based approach. FID measures how close a generated distribution is to a reference distribution. That is the objective a distribution-learning model is trained for, not the objective a planner driven by prompts or explicit constraints aims to achieve. The FID scores are therefore interpreted together with the geometric validity of Section 7.2; in fact, LayoutGPT trails ATISS in Table 4 but leads it in Table 5.

Fig. 6 presents qualitative experiment of scene synthesis results generated by the proposed method across the bedroom, living room, and dining room. The proposed method produces satisfactory placement results even under complex layout patterns and diverse object combinations. Notably, stable alignment and relational consistency are maintained not only in scenes with a large number of objects and increased geometric constraints, but also in simpler and more confined layouts. Overall, the results demonstrate that the proposed method generalizes effectively across various scene types, layouts, and object conditions through generalized learning.

7.2. Accuracy

In indoor scene synthesis, inter-object collisions or deviations beyond the floor boundary result in inaccurate or unnatural placements. Therefore, we conduct systematic experiments to evaluate the accuracy of the proposed framework in inferring stable and geometrically valid object placements.

7.2.1. OBB geometric validity

For each scene S, all furniture objects are represented using OBBs, and two types of constraint violations are defined: (1) When the OBB of an object extends beyond the valid floor region. (2) When the OBBs of two different objects overlap. Based on these definitions, the accuracy $A(S)$ of scene S is defined as follows:

$$A(S) = 1 - \frac{1}{|F|}\left(\sum_{i<j}\left|B_i \cap B_j\right| + \sum_i \left|B_i \cap \bar{F}\right|\right) \tag{9}$$

Here, B_i denotes the OBB of furniture object i contained in scene S and $\bar{F}$ represents the space outside the valid floor region. F is the valid floor region and $|F|$ is its area, and $|B_i \cap B_j|$ is the overlapping area of two OBBs. Each pair of objects is counted only once. Both penalty terms are expressed as a proportion of the floor area, and $A(S)$ is clamped to a minimum of 0. For an ideal scene that satisfies all constraints,

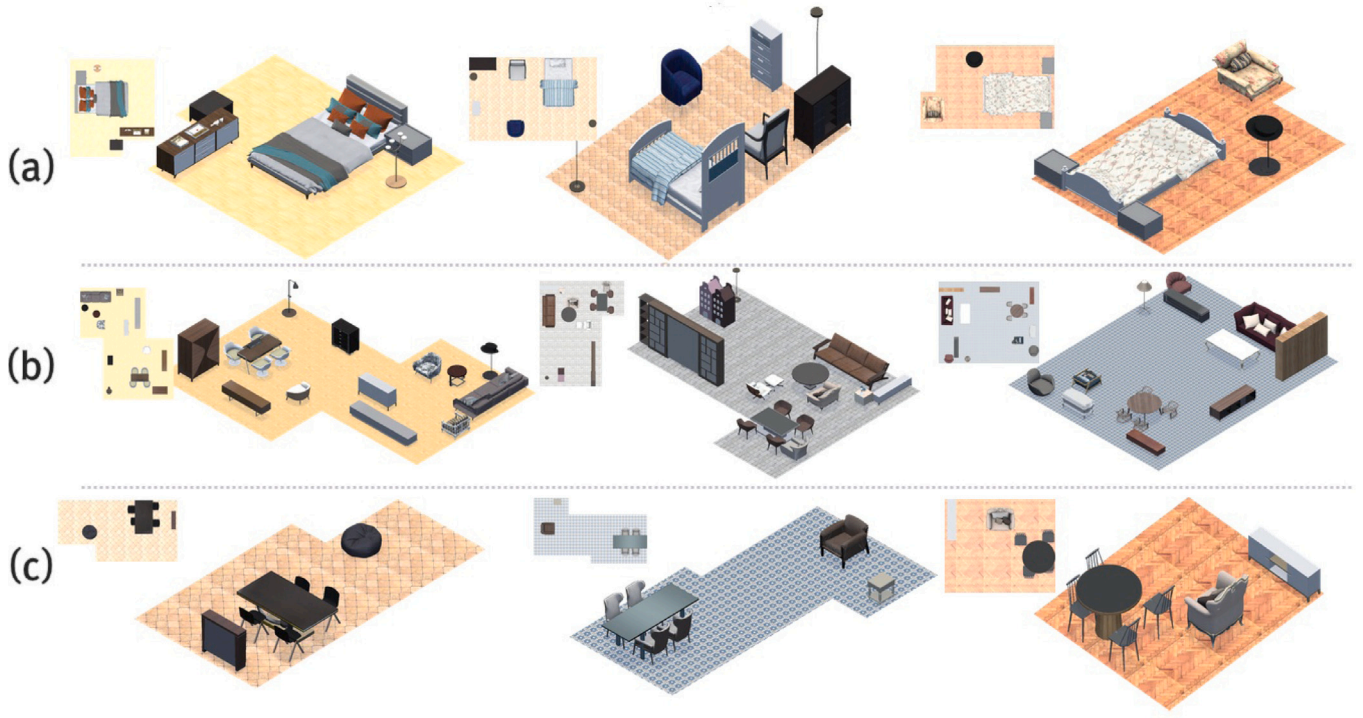


Fig. 6. Qualitative results for bedroom (a), living room (b), and dining room (c). The proposed method generates geometrically and semantically satisfactory placement results under complex and diverse layouts and object combinations.

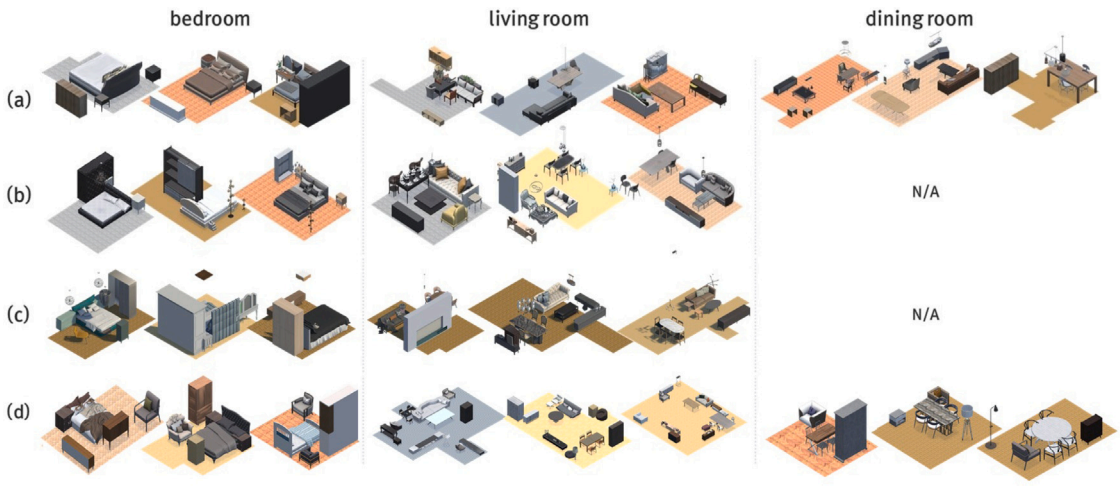


Fig. 7. 3D visualization examples of scenes generated by (a) ATISS [5], (b) LayoutGPT [7], (c) DiffuScene [6], and (d) the proposed method across different scene categories.

$A(S) = 1$. Fig. 7 visualizes representative scenes generated by existing methods and the proposed approach. Compared to prior methods, the proposed method exhibits negligible floor boundary violations. Fig. 8 zooms in on one living room scene per method and marks the most visible violations with boxes.

Table 5 presents the comparison results of OBB-based scene accuracy for scenes generated by each method. The proposed approach achieves high accuracy across most categories. In particular, it maintains stable accuracy even in the living room environment. These results consistently support that the policy learned through the proposed reward design effectively internalizes geometric constraints, in particular placement within the floor boundary, even under diverse layout patterns and object combinations. These results indicate that the proposed object-level policy scales effectively to cluttered environments with higher object counts, while maintaining geometric consistency. The released model of DiffuScene trained for the rearrangement setting generates the positions and orientations of objects without being conditioned on the floor boundary, and the part where its accuracy is reduced is mainly the floor-boundary term of Eq. (9) rather than the overlap term. The average penalty relative to the floor area is 0.053 for the floor-boundary term against 0.016 for the overlap term in the bedroom, and 0.126 against 0.036 in the living room. Accordingly, the gap in $A(S)$ between DiffuScene and the proposed method reflects this difference in conditioning on the floor boundary. Without the floor-boundary term, the proposed method is the most accurate only in the bedroom, as its overlap penalty in the living and dining rooms is higher than that of some of the compared methods.

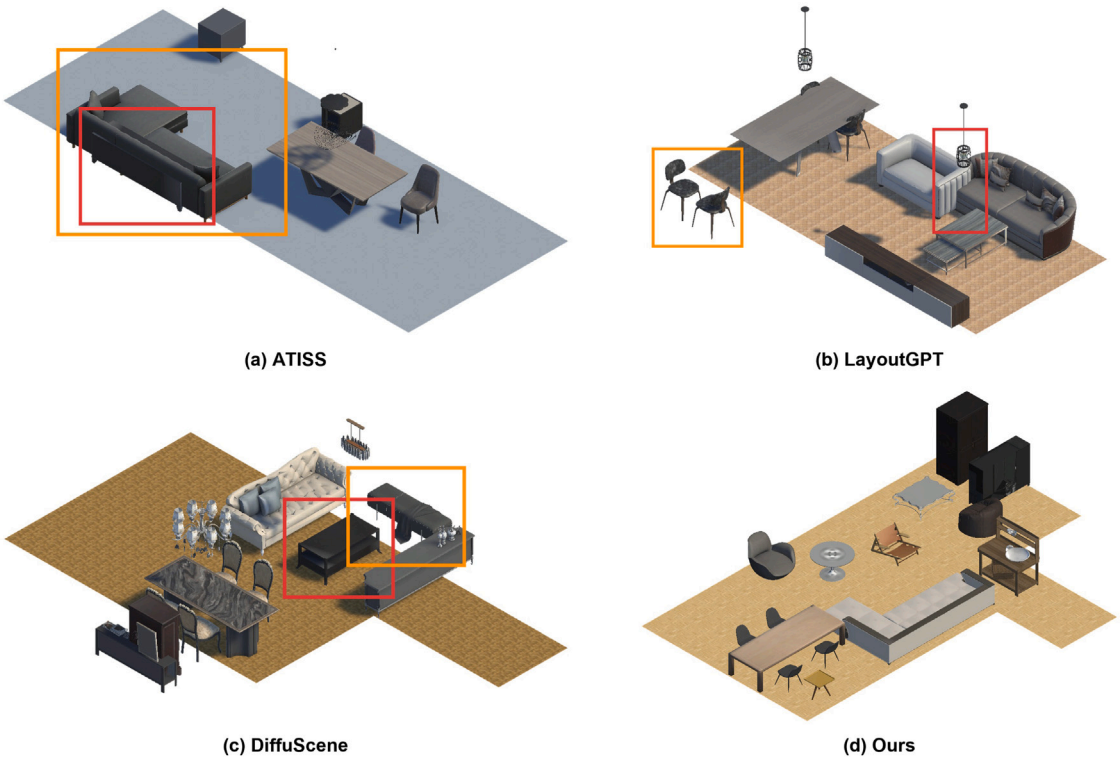


Fig. 8. One living room scene per method: (a) ATISS [5], (b) LayoutGPT [7], (c) DiffuScene [6], and (d) the proposed method. Red boxes mark overlaps between objects, and orange boxes mark objects placed outside the floor boundary. In (d) there is no overlap or floor-boundary violation larger than 0.06 m^2.

Table 5
OBB-based scene accuracy comparison between original 3D-FRONT [22] scenes and scenes generated by each method. Obj. is the average number of objects per scene, and w/o FB is the accuracy without the floor-boundary penalty.

Method	Bedroom			Living room			Dining room		
	Obj.	$A(S)$	w/o FB	Obj.	$A(S)$	w/o FB	Obj.	$A(S)$	w/o FB
3D-FRONT [22]	4.16	0.969	0.980	9.88	0.963	0.971	9.45	0.955	0.961
ATISS [5]	5.50	0.810	0.956	10.42	0.854	0.919	9.64	0.885	0.916
LayoutGPT [7]	5.07	0.951	0.973	8.41	0.899	0.955	–	–	–
DiffuScene [6]	4.08	0.932	0.984	10.15	0.838	0.964	–	–	–
Ours	4.07	0.981	0.989	16.99	0.939	0.950	8.99	0.867	0.882

Table 6
Quantitative comparison between the proposed method (Ours) and the random baseline across scene categories. The number in parentheses is the per-category step limit.

Category	Method	Completion	Success rate	Avg. Steps	Succ. Steps
Bedroom (500)	Ours	0.834	0.493	337.5	170.6
	Random	0.630	0.166	463.4	279.8
Living Room (1000)	Ours	0.964	0.623	616.3	384.3
	Random	0.543	0	1000	0
Dining Room (500)	Ours	0.957	0.824	237.0	180.6
	Random	0.506	0	500	0

7.2.2. Random baseline

To evaluate whether the reinforcement learning-based policy learns a structured constraint-satisfaction strategy beyond naive exploration, we compare the proposed method with a non-informative random baseline. The random baseline selects actions uniformly at random at each step without learning or rule-based reasoning. In Table 6, completion is the proportion of objects that satisfy their own placement condition across all evaluation episodes, and success rate is the proportion of episodes in which every object satisfies that condition within the step limit. The placement condition of a general object is to reach the wall-relative prior target within a tolerance, without overlap and within the floor boundary, and the condition of a child object is to reach the pair-wise target after its parent has settled. Avg. Steps is the average number of decision steps per episode, and Succ. Steps is the average number of decision steps measured over successful episodes only. Whether an object is satisfied is recorded at the step where the condition first holds and is not revised afterwards. The geometric validity of the final scene is measured separately in Section 7.2.1.

From the results shown in Table 6, the proposed method consistently achieves higher average completion ratio and success rate than the random baseline across all scene categories. In the bedroom category, *ours* achieves an average completion ratio of 0.834 and a success rate of 0.493, whereas the random baseline records an average completion ratio of 0.630 and a success rate of 0.166. This indicates that the proposed method not only reaches fully valid scene configurations

Table 7
OBB-based scene accuracy $A(S)$ for the four scenes of Fig. 10. The row order is the same as in Fig. 10, and the value in parentheses indicates the number of objects in the scene. In the $A(S)$ columns, Input is the $A(S)$ of Fig. 10(b), HAISOR is the $A(S)$ of Fig. 10(d), and Ours is the mean ± standard deviation over ten runs per scene (Fig. 10(e)). The columns headed parent–child overlap exempt are computed with the same $A(S)$ formula while excluding the overlap between parent–child pairs from the penalty.

Scene (number of objects)	$A(S)$			$A(S)$, parent–child overlap exempt		
	Input	HAISOR	Ours	Input	HAISOR	Ours
Living Room 1 (9)	0.977	0.993	0.984 ± 0.011	0.990	1.000	0.996 ± 0.005
Living Room 2 (10)	0.994	0.998	0.996 ± 0.007	0.999	1.000	0.999 ± 0.002
Bedroom 1 (6)	0.972	0.998	0.978 ± 0.007	1.000	1.000	0.998 ± 0.003
Bedroom 2 (11)	0.980	0.998	0.997 ± 0.003	0.982	0.998	0.999 ± 0.002

more frequently, but also maintains a higher proportion of furniture satisfying the target conditions within partially completed scenes, thereby forming more structurally consistent layouts. The performance gap becomes more pronounced in environments where the number of objects increases and placement constraints become more complex, such as living room and dining room. In the living room setting, the random baseline fails to generate a single successful scene during the entire evaluation period (success rate = 0), whereas the proposed method achieves a success rate of 0.623. A similar tendency is observed in the dining room. The random baseline again yields no successful cases and achieves an average completion ratio of only 0.506. In contrast, the proposed method records an average completion ratio exceeding 0.957 and a success rate above 0.824. Differences in generation efficiency are also evident. The random baseline typically terminates after reaching the maximum step limit (average steps = max step), indicating that most episodes end due to time constraints without satisfying the target conditions. In contrast, the proposed method shows significantly lower average successful steps compared to its overall average steps, suggesting that once valid structural conditions are met, the layout converges efficiently toward completion. Overall, comparison with the random baseline confirms that the proposed object-level reinforcement learning framework learns a structured placement strategy rather than relying on random exploration.

7.2.3. Ablation study

To analyze the relationship between the proposed reward components and scene layout quality in a stepwise manner, we conduct an ablation study. The full model is composed of three reward components: the accessibility reward, the prior reward for general objects, and the pairwise relationship reward for child objects. To evaluate the contribution of each reward term, we construct two ablation settings:

1. w/o accessibility reward: A configuration without the accessibility reward (r_{acc}).
2. w/o prior and pairwise rewards: A configuration without the prior and pairwise relationship rewards (r_{prior}, r_{pair}).

All ablation experiments are performed under the same training environment, observation configuration, action space, and learning hyperparameters as the full model, with only the reward function components modified. Fig. 9 illustrates the layout results generated by the full model using all reward components and by the ablation variants where each reward term is individually removed. When the accessibility reward is removed (Fig. 9(c)), frequent collisions between furniture objects are observed. When the prior and pairwise relationship rewards are removed (Fig. 9(d)), instability in wall distance and orientation alignment appears; large furniture such as cabinets are placed near the center of the room, and child objects such as chairs or side tables are positioned without considering their relative relationship to their corresponding parent objects, resulting in semantically meaningless placements. These ablation results indicate that the proposed reward components do not function independently; rather, they contribute to scene layout quality in a complementary manner when jointly integrated.

Table 8
Average generation time (seconds) per scene for each method. Each entry is the total generation time divided by the number of scenes produced.

Category	ATISS [5]	LayoutGPT [7]	DiffuScene [6]	Ours
Bedroom	11.29	2.97	28.74	15.43
Living Room	15.48	4.11	35.47	18.35
Dining Room	14.35	–	–	5.59

7.2.4. Comparison with existing reinforcement learning-based scene synthesis

To compare with existing reinforcement learning-based scene synthesis work, we conduct an experiment with HAISOR, the human-aware indoor scene optimization method proposed by Sun et al. [10]. HAISOR optimizes 3D indoor scene layouts based on human-aware objectives and rearranges furniture to preserve walkable space and support interactions between users and furniture, such as ensuring sufficient clearance to open and close cabinet doors. We used the input scenes from the accuracy experiments of the prior work as inputs to the proposed method. Fig. 10 compares the results of the prior work with those of the proposed method using the input scenes from HAISOR (Fig. 10(a, b)). To evaluate scene accuracy $A(S)$ under identical conditions, we reconstructed the image result of HAISOR (Fig. 10(c)) as a 3D scene (Fig. 10(d)) using our furniture models. Using the same models, we then applied the proposed method to the reconstructed input scene (Fig. 10(e)). The results show stable placements without visible overlap between objects. The proposed method produces different layouts from HAISOR because its general objects are encouraged to remain close to walls. Although our method does not explicitly model human affordance, this characteristic may help preserve walkable space. Table 7 presents the comparison based on the scene accuracy $A(S)$. Across the evaluated scenes, the scores ranged from 0.972–0.994 for the inputs, 0.993–0.998 for HAISOR [10], and 0.978–0.997 for the proposed method. Because the proposed method permits overlaps between parent and child objects, we also computed scene accuracy after excluding parent–child overlaps from the penalty. Although HAISOR relocates objects sequentially, the proposed method achieved high scene accuracy scores while placing all furniture concurrently. This comparison demonstrates that independent agents can produce geometrically stable scenes through concurrent placement actions.

7.3. Scalability

Efficiency in scene synthesis should be evaluated not only in terms of the average generation time for a single scene, but also in terms of the total processing time required in practical scenarios where a large number of scenes must be generated. In particular, in applications such as dataset construction, simulation-based training, or content creation involving complex or numerous scenes, the number of scenes generated within a given time period (throughput) can be a more important metric than the latency of individual scene generation. In this study, we analyze the generation efficiency of the proposed framework from two perspectives: single-scene latency and parallel scalability.

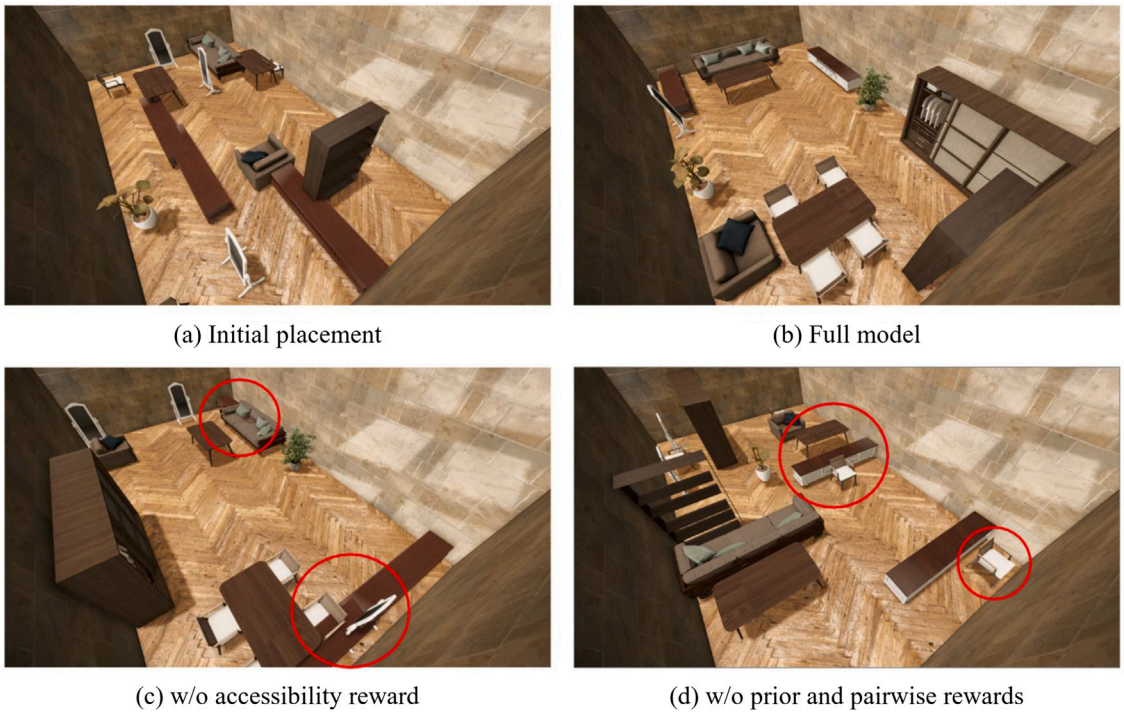


Fig. 9. Comparison of layout results under different reward configurations. (a) initial placement: the random state from which every run starts, (b) full model: all reward terms included, (c) w/o accessibility reward, (d) w/o prior and pairwise rewards.

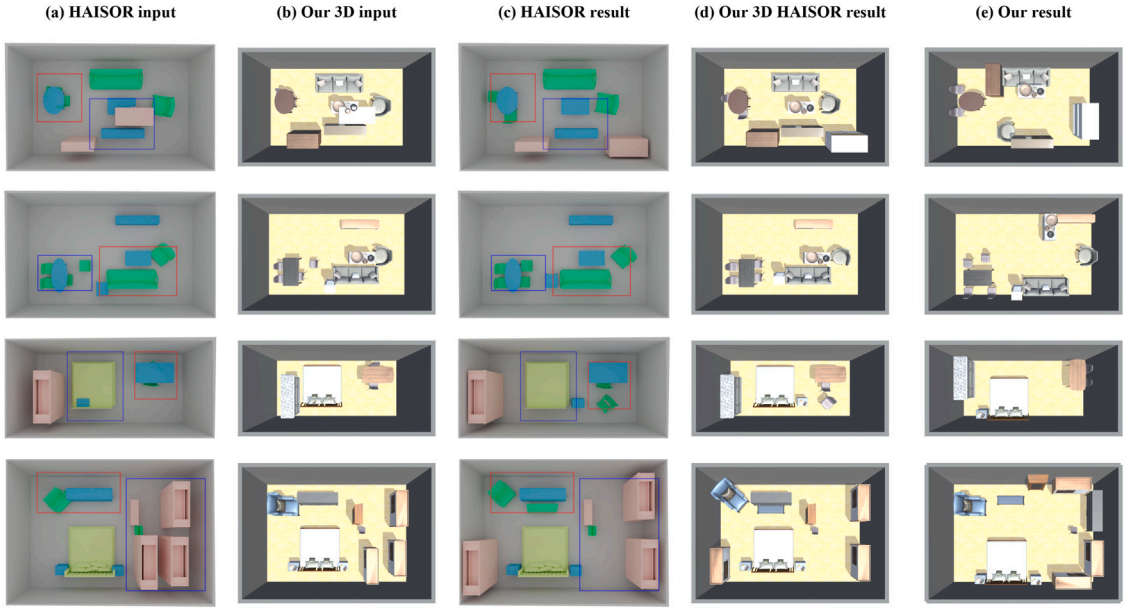


Fig. 10. The input scenes and result scenes used in HAISOR [10], reconstructed in our experimental environment together with our results. (a) the input scene of HAISOR [10], (b) the input scene reconstructed in our experimental environment from the input of HAISOR [10], (c) the result scene of HAISOR [10], (d) the output scene reconstructed in our experimental environment from the output of HAISOR [10], and (e) the result scene of ours inferred with our 3D input (b) as the input.

7.3.1. Scene latency

Table 8 compares the average time required to generate a single scene. All measurements in the table were conducted under the following hardware environment: Windows 11 64-bit with an Intel Core i9-13900 CPU, 32 GB RAM, and an NVIDIA GeForce RTX 4070 Ti GPU. Every entry in Table 8 was measured in the same manner. For each method, we ran the complete pipeline from beginning to end and divided the total generation time by the number of scenes produced. ATISS [5], LayoutGPT [7], and DiffuScene [6] were run using their publicly released pipelines without modification, while the proposed method generated 200 scenes sequentially without parallel execution.

Under the single-scene setting, ATISS and LayoutGPT exhibit relatively shorter generation times. This is because a complete scene can be produced through a single forward inference from the trained model. DiffuScene, in contrast, starts from random noise and completes a layout by repeating about 1000 denoising steps that gradually remove the

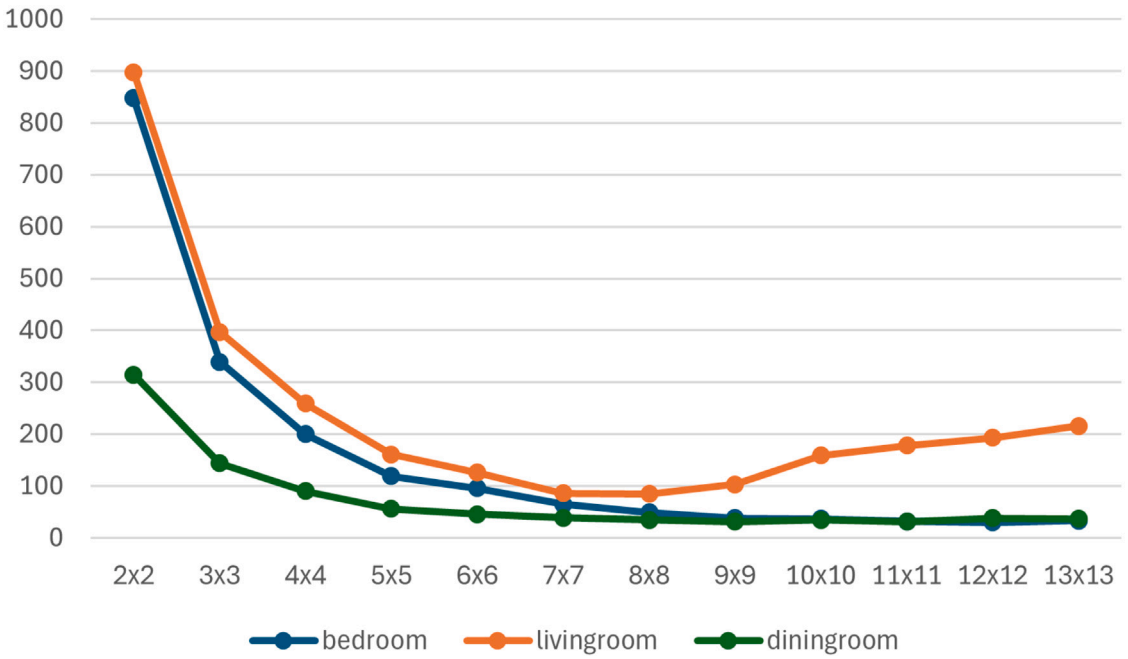


Fig. 11. Total time required to generate 200 scenes under different levels of parallel execution (2 × 2 to 13 × 13 grids). As the number of parallel scenes increases, the overall generation time decreases; beyond a certain scale, however, the reduction rate becomes more gradual due to scene complexity and hardware resource limitations.

noise, and therefore shows the largest value in Table 8. In contrast, the proposed method involves an optimization process in which multiple agents interact and gradually converge to a configuration that satisfies geometric constraints. The entries of the proposed method in Table 8 therefore grow with the number of decision steps an episode takes, rather than with the cost of a single inference. Furthermore, under certain initial conditions, episodes may terminate after reaching the maximum step limit without all objects fully satisfying the geometric constraints. Such non-convergent episodes contribute to an increase in the average generation time. The dining room entry is the smallest of the three because this category is the easiest for the trained policy. In Table 6 the dining room has the highest success rate (dining room 0.824, bedroom 0.493, living room 0.623) and settles in the fewest decision steps (dining room 237.0, bedroom 337.5, living room 616.3). Since all agents select their actions simultaneously, the episode time follows the number of steps until convergence rather than the number of objects.

7.3.2. Parallel throughput

The proposed scene synthesis framework enables the simultaneous generation of multiple scenes under a shared policy. Each scene is instantiated as an independent environment, and each agent selects its action concurrently through the policy of its own role. All scenes are therefore processed by the same two policy networks. Since there is no dependency between different scene instances, parallel scaling can be achieved without retraining or modifying the policy. To validate this property, we constructed scene grids ranging from 2 × 2 to 13 × 13 and measured the total time required to generate 200 scenes for each category. Fig. 11 illustrates the change in total generation time as the degree of parallelism increases.

Experimental results show that, across all categories, the total generation time decreases significantly as the level of parallelism increases. For example, in the bedroom category, generating 200 scenes required approximately 850 s under a 2 × 2 grid configuration. This time was reduced to under 100 s with parallel configurations of 6 × 6 or larger, and further decreased to approximately 31 s at 10 × 10 or higher. A similar trend was observed for the dining room category. These results indicate that total generation time does not scale linearly as "single-scene time × number of scenes." Instead, because multiple scenes are synthesized simultaneously within a single policy execution process, the average cost per scene decreases as the level of parallelism increases. In the living room category, where the number of objects is larger and interactions are more complex, total generation time also decreased substantially in the early stages of parallel scaling. However, beyond approximately 7 × 7 to 8 × 8 parallel configurations, the reduction rate became more gradual, and in some cases the total time increased slightly. This phenomenon is associated with GPU resource saturation, as the number of simultaneously processed agents grows rapidly with increasing parallel scenes. Therefore, the benefit of parallel scaling depends on both hardware capacity and scene complexity, and an optimal scaling point exists.

Overall, these results demonstrate that the proposed framework possesses structural scalability, allowing multiple scenes to be synthesized in parallel under a shared policy. By combining object-level policy execution with independent scene environments, the framework supports large-scale scene synthesis and is well-suited for applications that require efficient generation of numerous scenes simultaneously. This measurement confirms that the framework has an efficient structure in which the time taken to produce many scenes at once is not proportional to the number of scenes, and consequently shows the potential of reinforcement learning-based scene synthesis for parallel scalability.

8. Limitation and discussion

Beyond the comparison with HAISOR presented in Section 7.2.4, this study does not include direct quantitative comparisons with other reinforcement learning-based scene synthesis approaches. This omission is not merely due to differences in experimental environments, but rather stems from fundamental differences in problem formulation, agent architecture, and supervision source. Table 9 summarizes these differences across representative prior works.

As Table 9 clarifies, Scene Mover [16] solves a fundamentally different task: it requires both an initial and a target layout as input and returns a sequence of moves between them, so it has no protocol for

Table 9
Conceptual comparison between the proposed method and prior reinforcement learning-based scene layout works. The methods differ along problem formulation, agent architecture, action timing, policy structure, supervision source, observation modality, and generalization scope. These structural differences are the main reason a direct, like-for-like quantitative comparison is non-trivial; per-method justifications are given in the paragraph following the table.

Method	Problem formulation	Agent architecture	Action timing	Policy structure	Supervision/reward source	Observation & generalization
Scene Mover [16]	Move planning between a given initial and a given target layout (rearrangement, not synthesis)	Single-agent RL with Monte Carlo Tree Search (MCTS)	Sequential pick-move-place over a planning horizon	Trained per planning instance; no shared policy across rooms	Path/collision cost given a fixed target configuration	Top-down occupancy plus target layout; per-task scope
RLSS [9]	Sequential from-scratch placement, one object per step	Single-agent PPO with greedy action-space reduction	Sequential one-object-per-step; no per-object iterative refinement after placement	Trained per scene type with predefined design objectives	Hand-designed objective satisfaction signal	Global 2D occupancy state; trained per scene type
HAISOR [10]	Refinement of an already-arranged scene toward human affordance (not synthesis from scratch)	Single-agent Dueling Double DQN with MCTS over a hierarchical scene graph	Sequential edit operations on the scene graph	Trained per room type; relies on a pre-given scene graph	Human-aware rewards: free walkable space, part-mobility manipulation	Hierarchical scene graph as input; assumes a valid initial scene
Ours	From-scratch synthesis via iterative simultaneous convergence	Object-level reinforcement learning: one agent per furniture object, with role-based (general/child) policies	All agents act simultaneously at every step; layout converges through repeated interaction	Shared PPO policy per role (general/child), applied across layouts, room types, and object sets	Reference-scene priors: wall-relative for general agents, parent–child for child agents; collision avoidance	Object-local ray observation plus role-specific state; cross-layout and cross-room-type generalization

synthesizing a layout from scratch. RLSS [9] adopts a sequential one-object-per-step protocol with hand-designed objective rewards, so once an object is placed it cannot be revisited or refined; this is incompatible with our simultaneous, iterative-refinement protocol and makes per-step metrics non-comparable. HAISOR [10] consumes a hierarchical scene graph of an already arranged scene and outputs edit operations to improve human affordance. We showed in Section 7.2.4 that the proposed policy can also operate from such a scene; the two methods differ not in their initial scenes but in their optimization objectives. Across these works the differences are not only in implementation but in problem formulation, observation modality, and the unit of decision making, which together preclude controlled head-to-head comparison without redefining one of the two protocols. The primary objective of this study, accordingly, is not to compare convergence speed or reward magnitude under a fixed environment, but to verify whether a unified policy structure can consistently form stable placement strategies under varying scene conditions.

Beyond performance comparison in scene synthesis, this work also holds potential from a dataset construction perspective. As the usage of large-scale indoor scene datasets such as SUNCG [27] has become restricted, the need to identify alternative datasets or construct new ones has emerged. The proposed framework offers a structure that does not directly learn a global distribution, yet repeatedly generates diverse scene layouts based on object-type-specific prior information and reward design. In particular, through parallel execution, we demonstrate that large numbers of scenes can be synthesized efficiently. This suggests that the proposed framework can extend beyond a simple optimization technique and function as a 3D indoor scene data generation pipeline.

Nevertheless, several limitations remain. First, because the rotation is discretized into 90° increments, orientations that deviate from that increment cannot be represented by the current action space. Layouts that require furniture placements at various angles are therefore not generated, and an extension to a finer rotation increment is required.

Second, the current reward design is limited to physical collision avoidance and semantic relationship preservation between objects. Human-centered factors such as navigation flow, visibility, and usability are not explicitly incorporated, although prior work has shown that such factors can be encoded as part of the synthesis objective [28]. Future work should integrate virtual user models or behavioral simulations to extend the synthesis objective beyond geometric and semantic constraints toward interaction-aware layout design. Third, the evaluation in this paper is entirely quantitative. Whether the generated layouts possess human-level scene quality cannot be confirmed by the geometric and distributional metrics used here, and assessing it requires a user study that incorporates aesthetic considerations, or expert-based validation.

Fourth, the object set is not inferred by the framework but is part of the input. The policy only places the given objects; it does not decide which categories of objects, and how many of them, should be put in an empty room. Deciding that list from the size, shape, and purpose of the room is a separate problem, and a model that proposes such a list can be combined with this framework directly, since only the category list and the parent assignment of each child have to be exchanged. The parent–child relations are likewise not inferred by the framework but are given together with the input. On top of these relations the placement targets are defined between a child and its parent, while general objects among themselves, and children with different parents, are placed with respect to the wall without targets defined between them. A group that requires relations between objects that are not in a parent–child relation, such as a sofa, a coffee table, and a TV facing one another, is therefore not captured by the current reward, and extending the pairwise targets to such object pairs is required.

Fifth, the two policies were trained as independent learners that share parameters within the same role, and no comparison with centralized training was performed. Whether this choice is better than centralized training is a question to be confirmed by a comparison under the same conditions, and we leave it as future work.

9. Conclusion

This study proposes a novel framework that reinterprets indoor scene synthesis through object-level reinforcement learning. By establishing a one-to-one correspondence between reinforcement learning agents and furniture objects, the framework forms a scene layout through locally grounded interactions driven by reward and action

mechanisms. Based on role-specific reward design and scene generalization strategies, the proposed framework enables a single shared policy to flexibly adapt to diverse layouts and object combinations. Through systematic quantitative and qualitative experiments, we demonstrate that the method produces accurate scene layouts that simultaneously satisfy geometric constraints and the given parent–child relations. Furthermore, by validating the generation efficiency under parallel execution, we confirm the scalability of the proposed approach and its potential for extension into scene dataset construction. These findings highlight the broader applicability of object-level reinforcement learning beyond conventional scene synthesis tasks.

CRediT authorship contribution statement

Yunsik Cho: Writing – original draft, Project administration, Methodology, Formal analysis. **Jiwon Lee:** Writing – review & editing, Visualization, Validation, Data curation. **Jinmo Kim:** Writing – review & editing, Writing – original draft, Validation, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) in order to support language refinement and improve the clarity and precision of English expression. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was financially supported by Hansung University.

Data availability

Data will be made available on request.

References

[1] Yu L-F, Yeung S-K, Tang C-K, Terzopoulos D, Chan TF, Osher SJ. Make it home: automatic optimization of furniture arrangement. ACM Trans Graph 2011;30(4). http://dx.doi.org/10.1145/1964921.1964981.

[2] Kán P, Kaufmann H. Automated interior design using a genetic algorithm. In: Proceedings of the 23rd ACM symposium on virtual reality software and technology. 2017, http://dx.doi.org/10.1145/3139131.3139135.

[3] Weiss T, Litteneker A, Duncan N, Nakada M, Jiang C, Yu L-F, Terzopoulos D. Fast and scalable position-based layout synthesis. IEEE Trans Vis Comput Graphics 2019;25(12):3231–43. http://dx.doi.org/10.1109/TVCG.2018.2866436.

[4] Yang H, Zhang Z, Yan S, Huang H, Ma C, Zheng Y, Bajaj C, Huang Q. Scene synthesis via uncertainty-driven attribute synchronization. In: Proceedings of the IEEE/CVF international conference on computer vision. 2021, p. 5610–20. http://dx.doi.org/10.1109/ICCV48922.2021.00558.

[5] Paschalidou D, Kar A, Shugrina M, Kreis K, Geiger A, Fidler S. ATISS: Autoregressive transformers for indoor scene synthesis. In: Advances in neural information processing systems. 2021, http://dx.doi.org/10.48550/arXiv.2110.03675.

[6] Tang J, Nie Y, Markhasin L, Dai A, Thies J, Nießner M. Diffuscene: Denoising diffusion models for generative indoor scene synthesis. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024, http://dx.doi.org/10.1109/CVPR52733.2024.01938.

[7] Feng W, Zhu W, Fu T-j, Jampani V, Akula A, He X, Basu S, Wang XE, Wang WY. LayoutGPT: compositional visual planning and generation with large language models. In: Proceedings of the 37th international conference on neural information processing systems. 2023, http://dx.doi.org/10.48550/arXiv.2305.15393.

[8] Yang Y, Sun F-Y, Weihs L, VanderBilt E, Herrasti A, Han W, Wu J, Haber N, Krishna R, Liu L, et al. Holodeck: Language guided generation of 3d embodied ai environments. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024, p. 16227–37. http://dx.doi.org/10.1109/CVPR52733.2024.01536.

[9] Ostonov A, Wonka P, Michels DL. RLSS: A deep reinforcement learning algorithm for sequential scene generation. In: 2022 IEEE/CVF winter conference on applications of computer vision. 2022, p. 2723–32. http://dx.doi.org/10.1109/WACV51458.2022.00278.

[10] Sun J-M, Yang J, Mo K, Lai Y-K, Guibas LJ, Gao L. HAISOR: Human-aware indoor scene optimization via deep reinforcement learning. ACM Trans Graph (ToG) 2024;43(2). http://dx.doi.org/10.1145/3632947.

[11] Kán P, Kaufmann H. Automatic furniture arrangement using greedy cost minimization. In: 2018 IEEE conference on virtual reality and 3D user interfaces. 2018, p. 491–8. http://dx.doi.org/10.1109/VR.2018.8448291.

[12] Ritchie D, Wang K, Lin Y-A. Fast and flexible indoor scene synthesis via deep convolutional generative models. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019, p. 6182–90. http://dx.doi.org/10.1109/CVPR.2019.00634.

[13] Yang Y, Jia B, Zhi P, Huang S. PhyScene: Physically interactable 3D scene synthesis for embodied AI. In: Proceedings of conference on computer vision and pattern recognition. 2024, http://dx.doi.org/10.1109/CVPR52733.2024.01539.

[14] Sun F-Y, Liu W, Gu S, Lim D, Bhat G, Tombari F, Li M, Haber N, Wu J. Layoutvlm: Differentiable optimization of 3d layout via vision-language models. In: Proceedings of the computer vision and pattern recognition conference. 2025, p. 29469–78. http://dx.doi.org/10.1109/CVPR52734.2025.02744.

[15] Bai T, Bai W, Chen D, Wu T, Li M, Ma R. FreeScene: Mixed graph diffusion for 3D scene synthesis from free prompts. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2025, p. 5893–903. http://dx.doi.org/10.1109/CVPR52734.2025.00553.

[16] Wang H, Liang W, Yu L-F. Scene mover: Automatic move planning for scene arrangement by deep reinforcement learning. ACM Trans Graph 2020;39(6). http://dx.doi.org/10.1145/3414685.3417788.

[17] Wei QA, Ding S, Park JJ, Sajnani R, Poulenard A, Sridhar S, Guibas L. LEGO-Net: Learning regular rearrangements of objects in rooms. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2023, p. 19037–47. http://dx.doi.org/10.1109/CVPR52729.2023.01825.

[18] Wu M, Zhong F, Xia Y, Dong H. TarGF: Learning target gradient field to rearrange objects without explicit goal specification. 2023, arXiv:2209.00853.

[19] de Witt CS, Gupta T, Makoviichuk D, Makoviychuk V, Torr PHS, Sun M, Whiteson S. Is independent learning all you need in the StarCraft multi-agent challenge? 2020, arXiv:2011.09533.

[20] Yu C, Velu A, Vinitsky E, Gao J, Wang Y, Bayen A, Wu Y. The surprising effectiveness of PPO in cooperative multi-agent games. In: Proceedings of the 36th international conference on neural information processing systems. Red Hook, NY, USA: Curran Associates Inc.; 2022.

[21] Wen M, Kuba JG, Lin R, Zhang W, Wen Y, Wang J, Yang Y. Multi-agent reinforcement learning is a sequence modeling problem. In: Proceedings of the 36th international conference on neural information processing systems. Red Hook, NY, USA: Curran Associates Inc.; 2022.

[22] Fu H, Cai B, Gao L, Zhang L-X, Wang J, Li C, Zeng Q, Sun C, Jia R, Zhao B, et al. 3D-front: 3d furnished rooms with layouts and semantics. In: Proceedings of the IEEE/CVF international conference on computer vision. 2021, p. 10933–42. http://dx.doi.org/10.1109/ICCV48922.2021.01075.

[23] Fu H, Jia R, Gao L, Gong M, Zhao B, Maybank S, Tao D. 3D-FUTURE: 3D furniture shape with texture. Int J Comput Vis 2021;129(12):3313–37. http://dx.doi.org/10.1007/s11263-021-01534-z.

[24] Unity Technologies. Unity. Game development platform; 2025.

[25] Juliani A, Berges V-P, Teng E, Cohen A, Harper J, Elion C, Goy C, Gao Y, Henry H, Mattar M, Lange D. Unity: A general platform for intelligent agents. 2020, http://dx.doi.org/10.48550/arXiv.1809.02627, arXiv preprint arXiv:1809.02627.

[26] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. 2017, http://dx.doi.org/10.48550/arXiv.1707.06347, arXiv preprint arXiv:1707.06347.

[27] Song S, Yu F, Zeng A, Chang AX, Savva M, Funkhouser T. Semantic scene completion from a single depth image. In: Proceedings of 30th IEEE conference on computer vision and pattern recognition. 2017, http://dx.doi.org/10.1109/CVPR.2017.28.

[28] Qi S, Zhu Y, Huang S, Jiang C, Zhu S-C. Human-centric indoor scene synthesis using stochastic grammar. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2018, p. 5899–908. http://dx.doi.org/10.1109/CVPR.2018.00618.